Documentation Utilisateur

Fontanie Lili

Stetten-Pigasse Romane

Talleux Inès

Sommaire

1) Présentation générale

1.1) La machine

1.2) Le programme

2) Comment lancer le programme ?

3) Étapes de la simulation

3.1) Lecture du programme assembleur (`pgm.txt`)

3.1.1) Que faire en cas d'erreur ?

3.2) Conversion en code machine (`hexa.txt`)

3.3) Initialisation de la machine simulée

3.4) Chargement de `hexa.txt` dans la machine simulée

3.5) Simulation du processeur : boucle d'exécution

3.6) Entrées et sorties (interactions pendant l'exécution)

3.7) Arrêt du programme

1) Présentation générale

Ce programme est un simulateur de processeur, une machine fictive, qui permet d'exécuter un programme écrit en assembleur.

Le projet prend la forme d'une archive au format ZIP contenant :

- un répertoire avec le code-source du projet organisé plusieurs fichiers :
  - les fichiers pour la partie assembleur
    - les fichiers `nettoyage.c` et `traduction.c`
    - les fichiers d'entête `nettoyage.h` et `traduction.h`
  - les fichiers pour la partie simulation :
    - le fichier `simulation.c` qui implémente la machine simulée grâce aux fonctions représentant l'état du processeur, d'accès mémoire, de décodage des instructions, ainsi que d'exécution instruction par instruction ;
    - le fichier d'entête `simulation.h`
  - le fichier ** `main.c` qui gère l'initialisation de la machine, le chargement du programme, l'appel de la fonction d'exécution et la gestion de la fin de simulation ;
- un répertoire *docs* contenant :
  - cette documentation pour l'utilisateur *user.pdf*
  - une documentation pour le développeur *dev.pdf*, donnant des explications sur l'implémentation et indiquant quelles ont été les difficultés rencontrées au cours du projet ;
  - un exécutable C dont le nom est `simulateur`.

1.1) La machine

La machine est simulée par la structure `Machine` qui comprend une mémoire de 64 kilo-octets, adressée de 0 à 65535, représentée par un tableau de caractères. Dans cette mémoire seront stockés le programme, à partir de l'adresse 0, ainsi que les données de ce programme.

Le processeur représenté par la structure comprend les registres suivants :

- `PC` : dans ce registre se trouve l'adresse de la prochaine instruction à exécuter ;
- `r0 à r31` : 32 registres généraux de 32 bits chacun. `r0` est toujours égal à 0, même après une instruction le modifiant ;
- un registre d'état : ce registre comprend 3 bits, les bits Z, C et N. Ils sont mis à jour après l'exécution d'une instruction, suivant le résultat de l'instruction. Z est mis à 1 si le résultat est nul, à 0 sinon ; C est mis à 1 s'il y a une retenue, à 0 sinon et N est la recopie du bit de poids fort indiquant le signe du résultat.

1.2) Le programme

Le programme réalise :

- La conversion d'un programme assembleur (fichier `pgm.txt`) en instructions machine codées en hexadécimal (fichier `hexa.txt`)
- Le chargement de ces instructions en mémoire
- La simulation de l'exécution instruction par instruction, comme le ferait un processeur ;

L'utilisateur lance le projet une seule fois au début. Ensuite, il interagit lors des éventuelles instructions d'entrée `in`.

2) Comment lancer le programme ?

Un compilateur C compatible C99 est nécessaire, par exemple *gcc*.

L'utilisateur lance le programme depuis un terminal avec :

```
./simulateur pgm.txt
```

- `simulateur` : l'exécutable
- `pgm.txt` : le fichier texte contenant le programme assembleur

3) Étapes de la simulation

3.1) Lecture du programme assembleur (`pgm.txt`)

Rappel des opérateurs acceptés : `or` , `and` , `xor` , `add` , `sub` , `mul` , `div` , `shr` , `ldb` , `ldh` , `ldw` , `stb` , `sth` , `stw` , `jmp` , `jzs` , `jcs` , `jcc` , `jnc` , `in` , `out` , `rnd` , `hlt`

Chaque **ligne** de `pgm.txt` ne doit **pas dépasser 255 caractères**. Comme toute instruction valide est largement plus courte que ce nombre, vous ne devriez pas rencontrer de problème … à moins de faire exprès.

Les **étiquettes** en début de ligne sont **optionnelles et autorisées**, elles doivent être constituées **uniquement de lettres et de chiffres** (autres caractères, espaces non acceptés) et ne peuvent **pas** être :

- un nom de registre valide (de r0 à r31)
- un nombre décimal (nom composé uniquement de chiffres)
- un nombre hexadécimal (nom commençant par h et suivi d'un nombre en hexadécimal, exemples : `h60A` ou `hbbb`)

Leur **taille ne doit pas dépasser 63 caractères** et l'étiquette ne peut **pas être vide**

Une erreur sera signalée si ces règles ne sont pas respectées.

Dans un premier temps, `pgm.txt` est nettoyé (s'il y a des espaces inutiles en début et fin de ligne) et les éventuelles étiquettes (si elles sont valides) sont enregistrées dans un tableau global avec l'adresse qui leur correspond.

Dans un second temps, la version nettoyée de `pgm.txt` est parcourue et notre programme vérifie que chaque instruction est correcte en terme de syntaxe et d'arguments.

3.1.1) Que faire en cas d'erreur ?

Message d'erreur	Cause possible	Solution
Vous utilisez un opérande inconnu à notre système.	Faute de frappe dans un opérateur, comme `addd` au lieu de `add` par exemple.	Vérifiez l'orthographe de votre instruction et assurez-vous qu'elle existe dans la liste des opérateurs.
Le nombre d'arguments fournis est invalide.	Vous avez fourni trop, ou trop peu d'arguments pour un opérateur donné. Par exemple : `add r1 r2` ou `hlt r1` .	Consultez le nombre d'arguments nécessaires à l'opérateur utilisé.
L'entrée fournie est trop longue.	Un mot dépasse la taille maximale du buffer (64 caractères)	Raccourcissez le nom de vos étiquettes ou vérifiez qu'il ne manque pas de séparateur entre deux mots.
Le registre fourni est invalide.	Le registre fourni n'existe pas. Par exemple : `r-1` ou `r32` . Une autre possibilité est que le format est incorrect, par exemple `r3xa` .	Utilisez uniquement les registres `r0` à `r31` inclus.
La valeur immédiate fournie est invalide.	La valeur est mal formatée (il manque le `#`), contient des lettres invalides, ou dépasse la capacité de 16 bits (>65 535)	Assurez-vous que la valeur commence par `#` et qu'elle est inférieure ou égale à 65 535
La taille de la sortie est trop petite pour contenir l'instruction.	Erreur qui n'est pas destinée à l'utilisateur : elle arrive si le développeur modifie la taille du buffer dans le code source et le rend trop petit.	Contactez me développeur pour qu'il corrige le code source.
La syntaxe de l'instruction est invalide.	Erreur de ponctuation : virgule au début de l'instruction, double virgule (`,,`), parenthèses manquantes pour `ld` / `st` ou caractères interdits.	Vérifiez la syntaxe de votre instruction. Exemple pour load : `ld r1, (r2)#0`
L'étiquette n'est pas définie.	Vous tentez de faire un saut vers une étiquette qui n'a pas été déclarée dans le code.	Déclarez l'étiquette de cette manière : `etiquette:` ou corrigez le nom dans l'instruction de saut.

3.2) Conversion en code machine (`hexa.txt`)

S'il n'y a pas eu d'erreurs durant la lecture, on convertit `pgm.txt` en hexadécimal :

Chaque ligne du fichier assembleur est divisée en morceaux : le nom de l'instruction, les registres et les opérandes. Puis, le programme code les différents champs de l'instruction sur 32 bits. Cela se présente comme suit :

Champ	Taille	Rôle
code opératoire	5 bits	type d'instruction
rd	5 bits	registre destination
rn	5 bits	registre source
Imm	1 bit	indique si on utilise un immédiat
src2	16 bits	registre ou valeur immédiate

Le nom de l'instruction est converti en code opératoire numérique, les registres sont convertis en nombres correspondant au numéro de registre et les valeurs immédiates sont limitées à 16 bits. Les différents champs sont ensuite placés dans les bits correspondants pour former le mot machine de 32 bits. Puis, l'instruction est convertie en 8 chiffres hexadécimaux et chaque ligne correspondant à une instruction machine est écrite dans le fichier `hexa.txt` .

3.3) Initialisation de la machine simulée

Avant l'exécution, la machine est remise à zéro grâce à la fonction `machine_zero` : tous les registres sont mis à 0 (`R[0] à R[31]`) ; le compteur ordinal `PC` est mis à 0 ; les indicateurs (`Z, C, N`) sont mis à 0 ; toute la mémoire (64 Ko) est initialisée à 0. Cela permet d'éviter que les registres, la mémoire et les indicateurs d'état contiennent des valeurs indéfinies.

3.4) Chargement de `hexa.txt` dans la machine simulée

Le fichier `hexa.txt` est ensuite lu ligne par ligne grâce à la fonction `charger_hexa_txt()` . Pour chaque instruction, le simulateur lit les 8 caractères hexadécimaux, les convertit en un entier sur 32 bits = 4 octets et écrit l'instruction en mémoire grâce à la fonction `ecrire_mem32` à l'adresse :

- instruction 0 : adresse 0
- instruction 1 : adresse 4
- instruction 2 : adresse 8

etc.

3.5) Simulation du processeur : boucle d'exécution

Après le chargement, le simulateur entre dans une boucle dont chaque itération appelle la fonction `execute_instruction()` qui permet l'exécution d'une instruction machine et la mise à jour de l'état interne de la machine fictive. Plus précisément, via cette fonction, le simulateur :

- Lit l'instruction en mémoire à l'adresse `PC` via la fonction `lire_mem32` qui permet de recomposer une instruction de 4 octets précédemment stockée en mémoire ;
- Grâce à la fonction `decode` , la décode pour obtenir :
  - Le code opération (type d'instruction)
  - Les registres concernés (registre destination et registre source)
  - L'opérande `S` (registre ou valeur immédiate). Cet opérande pouvant être négatif, l'extension de signe permettant de préserver le signe lors de la conversion vers un type plus large se fait explicitement par la fonction `ext_signe16` .
- Exécute l'instruction :
  - Opérations logiques : `or, and, xor`
  - Opérations arithmétiques : `add, sub, mul, div`
  - Décalages : `shr` (droite ou gauche selon le signe de `S`)
  - Transferts mémoire : `ldb, ldh, ldw, stb, sth, stw`
  - Sauts : `jmp, jzs, jzc, jcs, jcc, jns, jnc`
  - Entrées/sorties : `in, out`
  - Aléatoire : `rnd`
- Met à jour les flags (`Z, N, C`) si nécessaire via la fonction `set_ZN` ;
- Met à jour le `PC` : par défaut `PC = PC + 4` ou bien `PC` prend une autre valeur si l'instruction est un saut.

Cette boucle continue jusqu'à l'instruction `hlt` qui indique la fin du programme.

3.6) Entrées et sorties (interactions pendant l'exécution)

Instruction `in` :

Quand le programme exécute `in`, le simulateur affiche :

[IN] Entrez un entier :

L'utilisateur tape un entier, qui est stocké dans un registre.

Instruction `out` :

Quand le programme exécute `out`, le simulateur affiche :

[OUT] valeur signée

3.7) Arrêt du programme

L'exécution s'arrête lorsque l'instruction `hlt` est exécutée. Le simulateur affiche ensuite un message final avec l'adresse contenue dans le PC final en hexadécimal, par exemple :

```
Fin simulation. PC final=0x00A4
```